

The *Custody Problem* in Enterprise Agentic AI

Provider privacy promises are necessary. They are not enough. The deeper question is whether the third-party agentic runtime itself becomes a privileged vulnerability: who controls what the agent can know, remember, retrieve, and do?

The core claim: an enterprise is only as secure as the agentic runtime that holds custody of its operational reasoning. When that runtime is third-party and off-the-shelf, the agent itself becomes a privileged vulnerability. Silent context bleed is one failure mode; the deeper risk is surrendering control over context assembly, memory, tool execution, policy defaults, and domain boundaries.

- Runtime sovereignty
- Third-party agent risk
- Domain-scoped reasoning
- Explicit publication

AUTHOR

Henok Ghebrechristos, PhD

FORMAT

Visual overview PDF

AUDIENCE

AI platform, security, architecture



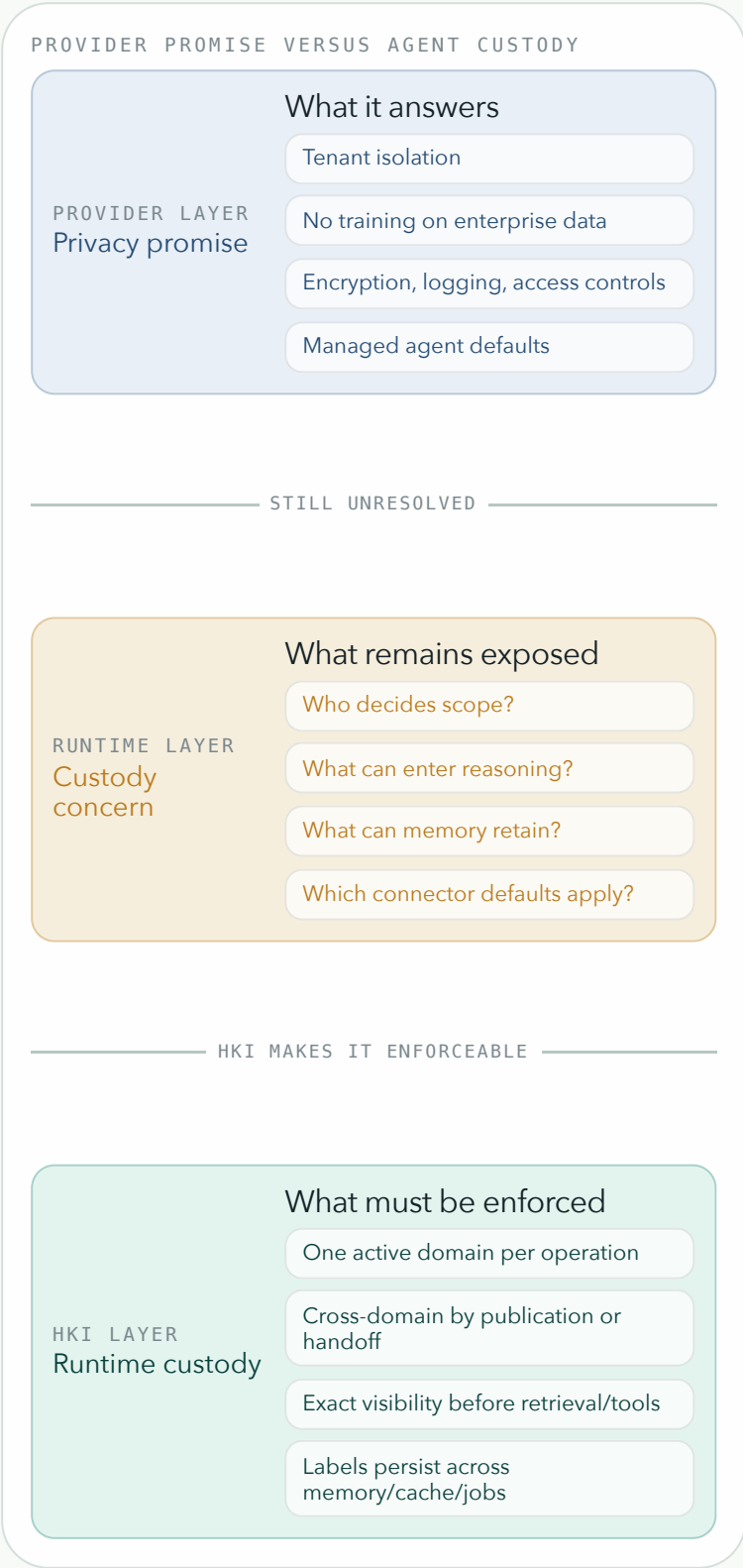
The security boundary has moved from training data to the agentic runtime itself.

No-training guarantees answer an important question. Off-the-shelf enterprise agents add a second one: who controls the runtime that assembles knowledge, invokes tools, stores memory, and decides what the agent can do?

A provider can truthfully say, "We do not train on your enterprise data," while its managed agent still retrieves enterprise documents, injects internal context, calls tools, stores memory, routes requests, logs telemetry, and learns operational preferences during inference.

That is the risk enterprises accept when they adopt managed agentic platforms such as Gemini Enterprise, ChatGPT Enterprise, or similar bundled agents out of the box. The platform is no longer just a model endpoint. It becomes a privileged execution layer inside the enterprise's work.

Context bleed remains dangerous, but it is downstream of custody: the runtime may decide what it can see before the enterprise sets an operation-level boundary. HKI does not block legitimate cross-domain work; executive legal/logistics answers or pharmacy order lookups are composed from separately scoped operations, then joined through publication, handoff, or audited provenance.



The custody problem appears when the agent runtime becomes a privileged black box.

The UI can look scoped while the execution path underneath quietly broadens through managed connectors, caches, graph traversal, accumulated memory, and tool routing.

01

Third-party agent as vulnerability

The managed agent is granted context, memory, connectors, and tool authority. If custody is external, the agent becomes part of the attack surface.

02

Context bleed

Retrieval, caches, graph edges, files, and conversations can inject adjacent-domain knowledge unless visibility is exact.

03

Memory and tool drift

Agent memory, tool routing, logs, and workflow preferences become hidden operational knowledge without durable labels.

04

Proof gap

Security claims remain promises unless every boundary is testable through conformance cases, probes, and release evidence.

The agent platform, not just the model, is the dependency.

Enterprise buyers often think they are purchasing model access. In practice, they are depending on a managed agentic substrate: memory, tool routing, retrieval, permissions, evaluation, logs, and workflow execution.

THE AGENTIC REASONING SURFACE

Retrieval

Documents, chunks, indexes, vector filters.

Memory

Preferences, vocabulary, escalation paths.

Tools

Actions, routing, privileges, workflow habits.

Cache

Prior answers, semantic hits, query traces.

Telemetry

Failures, retries, latency tradeoffs, operations.

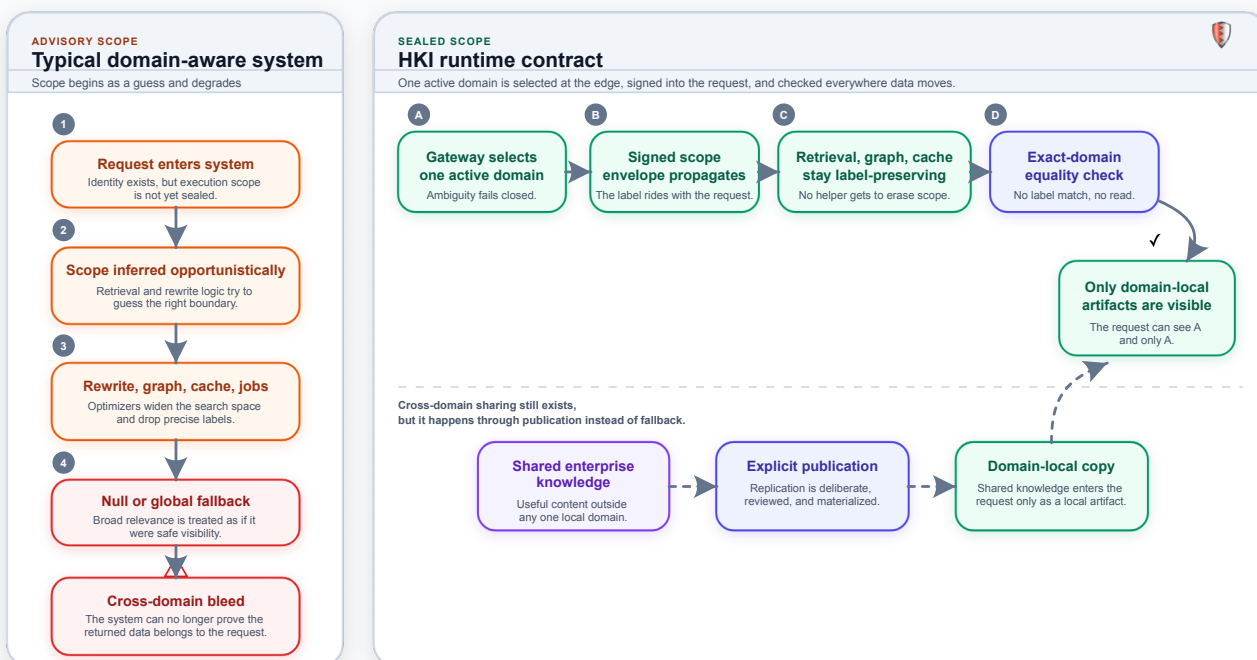
None of these layers has to become model-training data to become strategic signal. Whoever controls the agentic runtime controls what the agent sees, remembers, and does.

Conventional scope drifts. HKI treats scope as a sealed execution label.

The contrast is simple: advisory scope starts as an intention and can erode. Runtime custody keeps the active domain attached to every operation.

The flow contrast at a glance

Conventional systems often weaken scope as they optimize.
HKI preserves one active domain from the gateway to every downstream decision.



HKI selects one active domain for each runtime operation, propagates a signed scope envelope, enforces exact-domain equality, and limits visibility to domain-local artifacts plus explicitly published copies.

Switching costs grow quietly. By the time they are obvious, exit may be theoretical.

In cloud, dependency was compute and managed services. In agentic AI, dependency is reasoning, memory, and workflow execution.

YEAR 0

Off-the-shelf agent

Easy to adopt. The provider looks like a productivity vendor.

YEAR 1

Workflows accrue

Prompts, retrieval indexes, memories, and fine-tunes accumulate.

YEAR 2

Agents embed

Agents are inside daily operations. Exit now means re-engineering work.

YEAR 4

Runtime owns memory

Switching is technically possible, but operationally painful.

The danger does not require villains. It requires a privileged runtime you do not control.

Wrong hands is less a movie scene than an ordinary platform reality: a managed agent with connector reach, memory, tool authority, pricing power, product withdrawals, policy shifts, and silent defaults.

THE TECHNOFEUDAL ENDGAME

If an enterprise's reasoning, memory, and institutional habits are mediated by a provider-controlled agentic runtime, the provider becomes the de facto operator of operational intelligence. The enterprise keeps its legal shell, but the economic substance of decision-making migrates into an agent layer it does not control.

Concentration risk map

Pricing power

Leaving is expensive because memory and workflows live in the runtime.

Strategic withdrawal

Use cases, regions, or industries can lose support.

Ownership changes

Partnerships and incentives can change faster than enterprise systems migrate.

Silent defaults

Retention, logging, routing, and memory settings drift over time.

Move from provider trust to an enforceable runtime contract.

The answer is not to reject frontier providers or managed agents. The answer is to keep architectural authority over scope, memory, tools, and publication inside the enterprise before any third-party agent is allowed to act.

HERMETIC KNOWLEDGE ISOLATION - V1

GATEWAY

Choose one active domain

The edge decides the active domain for each runtime operation before retrieval, memory, tools, or model calls begin.

RUNTIME

Preserve labels everywhere

Artifacts, cache entries, memories, embeddings, tool calls, and outputs keep provenance and scope.

PUBLICATION

Make sharing explicit

Cross-domain knowledge moves through reviewed, auditable publication, not silent fallback or hidden retrieval.

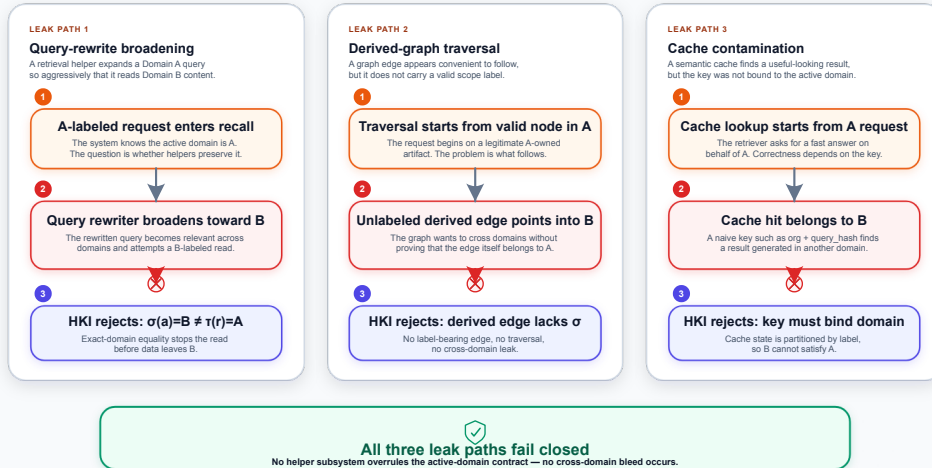
Every artifact belongs to one domain. Every runtime operation runs in one active domain. Cross-domain answers are composed through explicit publication, handoff, or audited joins - never silent global fallback.

Security becomes credible when the boundary is falsifiable.

HKI is not a slogan. It is a set of runtime checks that can be attacked, probed, and released with evidence.

Three leak paths, all blocked

HKI does not rely on good intentions from helper subsystems.
Each attempted boundary violation is rejected against the active-domain contract.



The recurring failures are query-rewrite broadening, derived graph bleed, and cache contamination. HKI blocks all three for the same reason: the system must preserve the active domain through every transformation, not only at the final read.

- Missing scope is rejected.
- Unauthorized active domains are rejected.
- Cross-domain graph edges are not traversed.
- Cache keys cannot omit the active domain.
- Publication creates new domain-labeled copies.

Build the boundary while it still belongs to you.

No enterprise adopts runtime custody in one cutover. The practical path is a staged migration from inventory to enforcement to ongoing evidence.

NEAR-TERM ROLLOUT SEQUENCE

01 AUDIT

Find null-scoped artifacts, weak cache keys, and endpoints that answer without explicit domain.

02 LABEL

Ensure every artifact type carries exactly one domain identity.

03 PROPAGATE

Select and sign active domain at the edge; reject contradictory scope downstream.

04 ENFORCE

Turn null-scope and non-preserving paths into readiness failures.

05 EVIDENCE

Add observability, adversarial tests, and release proofs that the boundary held.

BOTTOM LINE

Isolation cannot remain a provider promise. It has to become architecture.

The enterprises that remain sovereign in the agentic era will define what belongs where, what can be retrieved, what can be remembered, what can be shared, and what must be explicitly published before it crosses a domain boundary.

THE FINAL TEST

If a runtime operation can answer without an active domain, the boundary is a claim. If it cannot, the boundary is architecture.